TASK 9: SALES AND CUSTOMER ANALYTICS SYSTEM

STUDENT DETAILS

Name: Divya. V
Roll No: E25AI111
Department: B.Sc Computer Science With AI

SCENARIO

The Artisan Crumb Bakery & Pastry Management System wants to analyze its sales data to understand customer purchasing behavior, bakery item performance, and revenue trends. The management team needs reports to identify top customers, best-selling bakery items, and profitable categories.

1. Apply SQL Aggregate Functions

Functions Covered

Function	Purpose
COUNT()	Counts number of records
SUM()	Calculates total value
AVG()	Finds average value
MIN()	Finds minimum value
MAX()	Finds maximum value

2. Generate Total Sales Reports

Task

1. Total number of orders placed.

```
SQL> SELECT COUNT(*) AS Number_Of_Orders_Placed
  2  FROM Orders;

NUMBER_OF_ORDERS_PLACED
-----------------------
                      5
```

2. Total revenue generated from sales.

```
SQL> SELECT SUM(Total_Amount) AS Total_Revenue
  2  FROM Orders;

TOTAL_REVENUE
-------------
         5260
```

3. Average order value.

```
SQL> SELECT ROUND(AVG(Total_Amount), 2) AS Average_Order_Value
  2  FROM Orders;

AVERAGE_ORDER_VALUE
-------------------
               1052
```

4. Highest and lowest order amount.

```
SQL> SELECT MAX(Total_Amount) AS Highest_Order,
  2         MIN(Total_Amount) AS Lowest_Order
  3  FROM Orders;

HIGHEST_ORDER LOWEST_ORDER
------------- ------------
         1450          720
```

5. Total sales for a specific period.

```
SQL> SELECT SUM(Total_Amount) AS Total_Revenue
  2  FROM Orders
  3  WHERE Order_Date BETWEEN TO_DATE('2026-01-01', 'YYYY-MM-DD')
  4                       AND TO_DATE('2026-12-31', 'YYYY-MM-DD');

TOTAL_REVENUE
-------------
         5260
```

3. Customer Purchase Analysis

1. Total number of orders placed by each customer.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  3         COUNT(O.Order_ID) AS Total_Orders
  4  FROM Customer C
  5  LEFT JOIN Orders O ON C.Customer_ID = O.Customer_ID
  6  GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
  7  ORDER BY Total_Orders DESC;

CUSTOMER_ID          CUSTOMER_NAME                                TOTAL_ORDERS
-------------------- ---------------------------------------- ------------
C002                 Jane Smith                                              1
C001                 John Doe                                                1
C004                 Bob White                                               1
C005                 Charlie Green                                           1
C003                 Alice Brown                                             1
C006                 David Miller                                            0

6 rows selected.
```

2. Total amount spent by each customer.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  3         NVL(SUM(O.Total_Amount), 0) AS Total_Amount_Spent
  4  FROM Customer C
  5  LEFT JOIN Orders O ON C.Customer_ID = O.Customer_ID
  6  GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
  7  ORDER BY Total_Amount_Spent DESC;

CUSTOMER_ID          CUSTOMER_NAME                            TOTAL_AMOUNT_SPENT
-------------------- ---------------------------------------- ------------------
C003                 Alice Brown                                            1450
C004                 Bob White                                              1250
C005                 Charlie Green                                           950
C001                 John Doe                                                890
C002                 Jane Smith                                              720
C006                 David Miller                                              0

6 rows selected.
```

3. Average spending per customer.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  3         ROUND(NVL(AVG(O.Total_Amount), 0), 2) AS Average_Spending
  4  FROM Customer C
  5  LEFT JOIN Orders O ON C.Customer_ID = O.Customer_ID
  6  GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
  7  ORDER BY Average_Spending DESC;

CUSTOMER_ID          CUSTOMER_NAME                            AVERAGE_SPENDING
-------------------- ---------------------------------------- ----------------
C003                 Alice Brown                                          1450
C004                 Bob White                                            1250
C005                 Charlie Green                                         950
C001                 John Doe                                              890
C002                 Jane Smith                                            720
C006                 David Miller                                            0

6 rows selected.
```

4. Customers with maximum purchase amount.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  3         SUM(O.Total_Amount) AS Total_Purchase_Amount
  4  FROM Customer C
  5  JOIN Orders O ON C.Customer_ID = O.Customer_ID
  6  GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
  7  HAVING SUM(O.Total_Amount) = (
  8      SELECT MAX(Total_Spending)
  9      FROM (
 10          SELECT SUM(Total_Amount) AS Total_Spending
 11          FROM Orders
 12          GROUP BY Customer_ID
 13      )
 14  );

CUSTOMER_ID          CUSTOMER_NAME                          TOTAL_PURCHASE_AMOUNT
-------------------- ---------------------------------------- --------------------
C003                 Alice Brown                                            1450
```

5. Customers with fewer purchases.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  3         COUNT(O.Order_ID) AS Number_Of_Orders
  4  FROM Customer C
  5  LEFT JOIN Orders O ON C.Customer_ID = O.Customer_ID
  6  GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
  7  HAVING COUNT(O.Order_ID) <= 1
  8  ORDER BY Number_Of_Orders;

CUSTOMER_ID          CUSTOMER_NAME                             NUMBER_OF_ORDERS
-------------------- ---------------------------------------- ----------------
C006                 David Miller                                            0
C001                 John Doe                                                1
C005                 Charlie Green                                           1
C003                 Alice Brown                                             1
C002                 Jane Smith                                              1
C004                 Bob White                                               1

6 rows selected.
```

4. Find Top Customers Based on Purchase Amount

1. Top 5 customers based on total spending.

```
SQL> SELECT *
  2  FROM (
  3      SELECT C.Customer_ID,
  4             C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  5             SUM(O.Total_Amount) AS Total_Spending
  6      FROM Customer C
  7      JOIN Orders O ON C.Customer_ID = O.Customer_ID
  8      GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
  9      ORDER BY Total_Spending DESC
 10  )
 11  WHERE ROWNUM <= 5;

CUSTOMER_ID          CUSTOMER_NAME                            TOTAL_SPENDING
-------------------- ---------------------------------------- --------------
C003                 Alice Brown                                        1450
C004                 Bob White                                          1250
C005                 Charlie Green                                       950
C001                 John Doe                                            890
C002                 Jane Smith                                          720
```

2. Customers with maximum number of orders.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  3         COUNT(O.Order_ID) AS Number_Of_Orders
  4  FROM Customer C
  5  JOIN Orders O ON C.Customer_ID = O.Customer_ID
  6  GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
  7  HAVING COUNT(O.Order_ID) = (
  8      SELECT MAX(Order_Count)
  9      FROM (
 10          SELECT COUNT(*) AS Order_Count
 11          FROM Orders
 12          GROUP BY Customer_ID
 13      )
 14  )
 15  ORDER BY Customer_Name;

CUSTOMER_ID          CUSTOMER_NAME                              NUMBER_OF_ORDERS
-------------------- ---------------------------------------- ----------------
C003                 Alice Brown                                             1
C004                 Bob White                                               1
C005                 Charlie Green                                           1
C002                 Jane Smith                                              1
C001                 John Doe                                                1
```

3. Frequent customers.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  3         COUNT(O.Order_ID) AS Number_Of_Orders
  4  FROM Customer C
  5  JOIN Orders O ON C.Customer_ID = O.Customer_ID
  6  GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
  7  HAVING COUNT(O.Order_ID) >= 2
  8  ORDER BY Number_Of_Orders DESC;

CUSTOMER_ID          CUSTOMER_NAME                            NUMBER_OF_ORDERS
-------------------- ---------------------------------------- ----------------
C001                 John Doe                                               3
```

4. High-value customers.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  3         COUNT(O.Order_ID) AS Number_Of_Orders,
  4         SUM(O.Total_Amount) AS Total_Spending
  5  FROM Customer C
  6  JOIN Orders O ON C.Customer_ID = O.Customer_ID
  7  GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
  8  HAVING SUM(O.Total_Amount) > 2000
  9  ORDER BY Total_Spending DESC;

CUSTOMER_ID          CUSTOMER_NAME                          NUMBER_OF_ORDERS TOTAL_SPENDING
-------------------- -------------------------------------- ---------------- --------------
C001                 John Doe                                              3           3590
```

5. Identify Best-Selling Products

1. Bakery items with maximum sales quantity.

```
SQL> SELECT P.Product_ID,
  2         P.Title,
  3         SUM(OD.Quantity) AS Total_Quantity_Sold
  4  FROM Product P
  5  JOIN Order_Details OD ON P.Product_ID = OD.Product_ID
  6  GROUP BY P.Product_ID, P.Title
  7  ORDER BY Total_Quantity_Sold DESC;

PRODUCT TITLE                          TOTAL_QUANTITY_SOLD
------- ------------------------------ -------------------
P001    Chocolate Fudge Cake                             3
P005    Black Forest Cake                                2
```

2. Bakery items generating highest revenue.

```
SQL> SELECT P.Product_ID,
  2         P.Title,
  3         SUM(OD.Quantity) AS Quantity_Sold,
  4         SUM(OD.Quantity * OD.Price) AS Total_Revenue
  5  FROM Product P
  6  JOIN Order_Details OD ON P.Product_ID = OD.Product_ID
  7  GROUP BY P.Product_ID, P.Title
  8  ORDER BY Total_Revenue DESC;

PRODUCT TITLE                          QUANTITY_SOLD TOTAL_REVENUE
------- ------------------------------ ------------- -------------
P001    Chocolate Fudge Cake                       3          2250
P005    Black Forest Cake                          2          1200
```

3. Least-selling bakery items.

```
SQL> SELECT P.Product_ID,
  2         P.Title,
  3         NVL(SUM(OD.Quantity), 0) AS Quantity_Sold
  4  FROM Product P
  5  LEFT JOIN Order_Details OD ON P.Product_ID = OD.Product_ID
  6  GROUP BY P.Product_ID, P.Title
  7  ORDER BY Quantity_Sold ASC;

PRODUCT TITLE                          QUANTITY_SOLD
------- ------------------------------ -------------
P006    Special Wedding Cake                       0
P003    Red Velvet Pastry                          0
P002    Sourdough Bread                            0
P004    Croissants                                 0
P005    Black Forest Cake                          2
P001    Chocolate Fudge Cake                       3

6 rows selected.
```

4. Bakery items requiring promotion.

```
SQL> SELECT P.Product_ID,
  2         P.Title,
  3         P.Price,
  4         I.Stock_Quantity,
  5         NVL(SUM(OD.Quantity), 0) AS Quantity_Sold
  6  FROM Product P
  7  JOIN Inventory I ON P.Product_ID = I.Product_ID
  8  LEFT JOIN Order_Details OD ON P.Product_ID = OD.Product_ID
  9  GROUP BY P.Product_ID, P.Title, P.Price, I.Stock_Quantity
 10  HAVING NVL(SUM(OD.Quantity), 0) <= 1
 11     AND I.Stock_Quantity > 0
 12  ORDER BY Quantity_Sold, P.Title;

PRODUCT TITLE                          PRICE STOCK_QUANTITY QUANTITY_SOLD
------- ------------------------------ ----- -------------- -------------
P004    Croissants                       200             30             0
P003    Red Velvet Pastry                450              5             0
P002    Sourdough Bread                  350             15             0
P006    Special Wedding Cake            1200             10             0
```

6. Perform Category-wise Sales Analysis

1. Total sales generated by each category.

```
SQL> SELECT C.Category_Name,
  2         SUM(OD.Quantity * OD.Price) AS Total_Sales
  3  FROM Category C
  4  JOIN Product P ON C.Category_ID = P.Category_ID
  5  JOIN Order_Details OD ON P.Product_ID = OD.Product_ID
  6  GROUP BY C.Category_ID, C.Category_Name
  7  ORDER BY Total_Sales DESC;

CATEGORY_NAME             TOTAL_SALES
------------------------- -----------
Celebration Cakes                3450
```

2. Number of products sold in each category.

```
SQL> SELECT C.Category_Name,
  2         SUM(OD.Quantity) AS Products_Sold
  3  FROM Category C
  4  JOIN Product P ON C.Category_ID = P.Category_ID
  5  JOIN Order_Details OD ON P.Product_ID = OD.Product_ID
  6  GROUP BY C.Category_ID, C.Category_Name
  7  ORDER BY Products_Sold DESC;

CATEGORY_NAME            PRODUCTS_SOLD
------------------------ -------------
Celebration Cakes                    5
```

3. Highest revenue-generating category.

```
SQL> SELECT C.Category_Name,
  2         SUM(OD.Quantity * OD.Price) AS Total_Revenue
  3  FROM Category C
  4  JOIN Product P ON C.Category_ID = P.Category_ID
  5  JOIN Order_Details OD ON P.Product_ID = OD.Product_ID
  6  GROUP BY C.Category_ID, C.Category_Name
  7  HAVING SUM(OD.Quantity * OD.Price) = (
  8      SELECT MAX(Category_Revenue)
  9      FROM (
 10          SELECT SUM(OD2.Quantity * OD2.Price) AS Category_Revenue
 11          FROM Category C2
 12          JOIN Product P2 ON C2.Category_ID = P2.Category_ID
 13          JOIN Order_Details OD2 ON P2.Product_ID = OD2.Product_ID
 14          GROUP BY C2.Category_ID
 15      )
 16  );

CATEGORY_NAME            TOTAL_REVENUE
------------------------ -------------
Celebration Cakes                3450
```

4. Average sales per category.

```
SQL> SELECT C.Category_Name,
  2         ROUND(AVG(OD.Quantity * OD.Price), 2) AS Average_Sales
  3  FROM Category C
  4  JOIN Product P ON C.Category_ID = P.Category_ID
  5  JOIN Order_Details OD ON P.Product_ID = OD.Product_ID
  6  GROUP BY C.Category_ID, C.Category_Name
  7  ORDER BY Average_Sales DESC;

CATEGORY_NAME             AVERAGE_SALES
------------------------- -------------
Celebration Cakes                  1150
```

7. Generate Business Analytics Reports

Report 1: Sales Performance Report

- Total Sales
- Number of Orders
- Average Order Value
- Highest Order Value

```
SQL> SELECT SUM(Total_Amount) AS Total_Sales,
  2         COUNT(Order_ID) AS Number_Of_Orders,
  3         ROUND(AVG(Total_Amount), 2) AS Average_Order_Value,
  4         MAX(Total_Amount) AS Highest_Order_Value
  5  FROM Orders;

TOTAL_SALES NUMBER_OF_ORDERS AVERAGE_ORDER_VALUE HIGHEST_ORDER_VALUE
----------- ---------------- ------------------- -------------------
       7960                7             1137.14                1500
```

Report 2: Customer Analytics Report

- Customer Name
- Number of Orders
- Total Spending
- Purchase Frequency

```
SQL> SELECT C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  2         COUNT(O.Order_ID) AS Number_Of_Orders,
  3         NVL(SUM(O.Total_Amount), 0) AS Total_Spending,
  4         COUNT(O.Order_ID) AS Purchase_Frequency
  5  FROM Customer C
  6  LEFT JOIN Orders O ON C.Customer_ID = O.Customer_ID
  7  GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
  8  ORDER BY Total_Spending DESC;

CUSTOMER_NAME                          NUMBER_OF_ORDERS TOTAL_SPENDING PURCHASE_FREQUENCY
-------------------------------------- ---------------- -------------- ------------------
John Doe                                              3           3590                  3
Alice Brown                                           1           1450                  1
Bob White                                             1           1250                  1
Charlie Green                                         1            950                  1
Jane Smith                                            1            720                  1
David Miller                                          0              0                  0

6 rows selected.
```

Report 3: Product Performance Report

- Bakery Item Name
- Quantity Sold
- Revenue Generated

```
SQL> SELECT P.Title AS Bakery_Item_Name,
  2         SUM(OD.Quantity) AS Quantity_Sold,
  3         SUM(OD.Quantity * OD.Price) AS Revenue_Generated
  4  FROM Product P
  5  JOIN Order_Details OD ON P.Product_ID = OD.Product_ID
  6  GROUP BY P.Product_ID, P.Title
  7  ORDER BY Revenue_Generated DESC;

BAKERY_ITEM_NAME               QUANTITY_SOLD REVENUE_GENERATED
------------------------------ ------------- -----------------
Chocolate Fudge Cake                       3              2250
Black Forest Cake                          2              1200
```

Report 4: Category Analysis Report

- Category Name
- Total Products Sold
- Total Revenue

```
SQL> SELECT C.Category_Name,
  2         SUM(OD.Quantity) AS Total_Products_Sold,
  3         SUM(OD.Quantity * OD.Price) AS Total_Revenue
  4  FROM Category C
  5  JOIN Product P ON C.Category_ID = P.Category_ID
  6  JOIN Order_Details OD ON P.Product_ID = OD.Product_ID
  7  GROUP BY C.Category_ID, C.Category_Name
  8  ORDER BY Total_Revenue DESC;

CATEGORY_NAME             TOTAL_PRODUCTS_SOLD TOTAL_REVENUE
------------------------- ------------------- -------------
Celebration Cakes                           5          3450
Artisan Breads                              3          1350
Morning Pastries                            2           600
```